

Lab 5: Review of Object-Oriented Programming

Ex1: Blood Type Inheritance and Compatibility Program

1. Enum RhFactor

```
enum RhFactor {  
    POSITIVE, NEGATIVE  
}
```

This defines the **Rhesus (Rh) factor**, which can be either **POSITIVE (+)** or **NEGATIVE (-)** — a critical part of blood typing.

2. Abstract Class BloodType

```
public class BloodType {  
    protected RhFactor rh;  
  
    public BloodType(RhFactor rh) {  
        this.rh = rh;  
    }  
  
    public RhFactor getRh() {  
        return rh;  
    }  
}
```

```

// Default type - should be overridden in subclasses
public String getType() {
    return "Unknown";
}

// Default logic - subclasses should override
public Set<String> possibleChildren(BloodType otherParent) {
    return new HashSet<>();
}

// Default logic - subclasses should override
public boolean canReceiveFrom(BloodType donor) {
    return false;
}

public String toString() {
    return getType() + (rh == RhFactor.POSITIVE ? "+" : "-");
}

public boolean equals(Object o) {
    if (o instanceof BloodType) {
        BloodType bt = (BloodType) o;
        return this.toString().equals(bt.toString());
    }
    return false;
}
}

```

Purpose: This is the **base class** for all blood types (A, B, AB, O).

It includes:

- rh: stores the Rh factor.
- getType(): overridden in subclasses to return the specific type (e.g., "A").
- possibleChildren(): overridden in subclasses to determine inheritance outcomes.
- canReceiveFrom(): overridden to implement transfusion compatibility.
- toString() & equals(): helps with readable display and comparisons.

3. Class Atype, Btype, Ctype, Abtype, Otype extends BloodType

```
class AType extends BloodType {
    public AType(RhFactor rh) {
        super(rh);
    }

    @Override
    public String getType() {
        return "A";
    }

    @Override
    public Set<String> possibleChildren(BloodType other) {
        Set<String> result = new HashSet<>();
        String otherType = other.getType();
        switch (otherType) {
            case "A":
                result.addAll(List.of("A", "O"));
                break;
            case "B":
                result.addAll(List.of("A", "B", "AB", "O"));
                break;
            case "AB":
                result.addAll(List.of("A", "B", "AB"));
                break;
            case "O":
                result.addAll(List.of("A", "O"));
                break;
        }
        return result;
    }

    @Override
    public boolean canReceiveFrom(BloodType donor) {
        return donor.getType().equals("A") || donor.getType().equals("O") &&
            (donor.getRh() == RhFactor.NEGATIVE || this.rh == RhFactor.POSITIVE);
    }
}
```

Inheritance Rules:

If one parent is A, the possible children types depend on the other parent:

- $A + A \rightarrow A, O$
- $A + B \rightarrow A, B, AB, O$
- $A + AB \rightarrow A, B, AB$
- $A + O \rightarrow A, O$

This is implemented in: “*public Set<String> possibleChildren(BloodType other)*”

Transfusion Rules:

A-type individuals can receive blood from:

- A and O types only.
- Rh rules:
 - If recipient is **Rh+**, they can receive from both Rh+ and Rh-.
 - If recipient is **Rh-**, they can receive **only from Rh-**.

Implemented in: “*public boolean canReceiveFrom(BloodType donor)*”

BType, ABType, OType Classes

These follow the same structure as AType but with their respective rules for inheritance and compatibility.

4. Person Class

Represents a **person with a name and blood type**.

```
class Person {
    String name;
    BloodType bloodType;

    public Person(String name, BloodType bloodType) {
        this.name = name;
        this.bloodType = bloodType;
    }

    public String getName() {
        return name;
    }

    public BloodType getBloodType() {
        return bloodType;
    }

    public String toString() {
        return name + " (" + bloodType + ")";
    }
}
```

6. BloodTypeProgram Class

This class contains the main method and utility functions.

```
public static BloodType createBloodType(String fullType) {
    String type;
    RhFactor rh;

    fullType = fullType.trim().toUpperCase();

    if (fullType.endsWith("+")) {
        rh = RhFactor.POSITIVE;
        type = fullType.substring(0, fullType.length() - 1);
    } else if (fullType.endsWith("-")) {
        rh = RhFactor.NEGATIVE;
        type = fullType.substring(0, fullType.length() - 1);
    } else {
        throw new IllegalArgumentException("Invalid format. Use types like A+, B-, AB+, O- etc.");
    }

    switch (type) {
        case "A":
            return new AType(rh);
        case "B":
            return new BType(rh);
        case "AB":
            return new ABType(rh);
        case "O":
            return new OType(rh);
        default:
            throw new IllegalArgumentException("Unknown blood type: " + type);
    }
}
```

Parses a string like "A+", "O-", etc., and returns the appropriate BloodType subclass object.

```
public static boolean isValidChild(BloodType father, BloodType mother, BloodType child) {
    Set<String> possible = father.possibleChildren(mother);
    return possible.contains(child.getType()) &&
        ((father.getRh() == RhFactor.NEGATIVE && mother.getRh() == RhFactor.NEGATIVE)
         ? child.getRh() == RhFactor.NEGATIVE
         : true);
}
```

Checks if the child blood type is valid based on the parents:

- Checks if the **child's ABO type** is in the set of possible children.

- Checks **Rh factor**: if both parents are Rh−, then the child must be Rh−.

```
public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    List<Person> people = new ArrayList<>();

    // Input
    System.out.print("Enter number of people: ");
    int n = sc.nextInt();
    sc.nextLine();
    for (int i = 0; i < n; i++) {
        System.out.print("Enter name: ");
        String name = sc.nextLine();
        System.out.print("Enter blood type (A+/A-/B+/AB-/...): ");
        String type = sc.nextLine();
        people.add(new Person(name, createBloodType(type)));
    }
}
```

Asks the user how many people they want to enter.

For each person:

- Asks for the person's **name** and **blood type** (e.g., A+, O-, etc.).
- Converts the blood type string to a proper BloodType object using createBloodType(type).
- Adds each person to a list called people.

```
// Part b: Check child validity
System.out.println("\n== Check Child Validity ==");
System.out.print("Father's blood type: ");
BloodType father = createBloodType(sc.nextLine());
System.out.print("Mother's blood type: ");
BloodType mother = createBloodType(sc.nextLine());
System.out.print("Child's blood type: ");
BloodType child = createBloodType(sc.nextLine());

boolean valid = isValidChild(father, mother, child);
System.out.println("Is child blood type valid? " + (valid ? "Yes" : "No"));
```

Prompts the user to input the **blood types of a father, mother, and child**.

Calls `isValidChild(father, mother, child)` to determine if the child's blood type is **genetically possible**.

- This checks both **ABO inheritance rules** and **Rh factor compatibility**:
 - If both parents are **Rh-**, then the child must also be **Rh-**.

Prints **"Yes"** if valid, otherwise **"No"**.

```
// Part c: Blood donors
System.out.println("\n== Blood Donor Matching ==");
System.out.print("Enter person name to find donors: ");
String targetName = sc.nextLine();
Person target = null;
for (Person p : people) {
    if (p.getName().equalsIgnoreCase(targetName)) {
        target = p;
        break;
    }
}

if (target == null) {
    System.out.println("Person not found.");
} else {
    System.out.println("Compatible donors for " + target.getName() + ":");
    for (Person donor : people) {
        if (!donor.equals(target) && target.getBloodType().canReceiveFrom(donor.getBloodType())) {
            System.out.println("- " + donor);
        }
    }
}
```

Asks for the **name of a person** from the earlier list (people) to find compatible blood donors.

Searches for that person in the list.

- If not found → print "Person not found".
- If found:
 - Iterate through all other people.
 - Check if each person can **donate blood to the target** using:
target.getBloodType().canReceiveFrom(donor.getBloodType())
 - Print the names of compatible donors.

EX2: Big Five Model – Psychological Trait Assessment

1. Abstract Class: Trait

```
class Trait {  
    protected String name;  
    protected int score;  
  
    public Trait(String name, int score) {  
        this.name = name;  
        this.score = score;  
    }  
  
    5 overrides  
    public String getDescription() {  
        return " ";  
    };  
  
    public String getRiskLevel() {  
        if (score >= 70) return "High";  
        if (score <= 30) return "Low";  
        return "Medium";  
    }  
  
    public int getScore() {  
        return score;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

Trait is an **abstract base class** representing a personality trait.

name: Name of the trait (e.g., "Openness").

score: Integer score (e.g., 70).

Constructor initializes trait name and score.

An **abstract method** that child classes must implement. It returns a textual description based on the score.

Converts the numeric score into a risk level: High, Low, or Medium.

Getters for score and name.

2. Concrete Subclasses for Each Trait

Each subclass extends Trait and implements getDescription() differently:
Openness:

```
class Openness extends Trait {
    public Openness(int score) {
        super( name: "Openness (O)", score);
    }

    @Override
    public String getDescription() {
        switch (getRiskLevel()) {
            case "High":
                return "Enjoys new ideas, curious, independent, creative.";
            case "Low":
                return "Conservative, prefers routine, resistant to new ideas.";
            default:
                return "No specific conclusion.";
        }
    }
}
```

Custom descriptions for high/low Openness scores.

Other traits follow the same pattern:

```
class Conscientiousness extends Trait {
    public Conscientiousness(int score) {
        super( name: "Conscientiousness (C)", score);
    }

    @Override
    public String getDescription() {
        switch (getRiskLevel()) {
            case "High":
                return "Hardworking, disciplined, responsible.";
            case "Low":
                return "Disorganized, irresponsible, lacks self-discipline.";
            default:
                return "No specific conclusion.";
        }
    }
}
```

```

class Extraversion extends Trait {
    public Extraversion(int score) {
        super( name: "Extraversion (E)", score);
    }

    @Override
    public String getDescription() {
        switch (getRiskLevel()) {
            case "High":
                return "Outgoing, energetic, sociable.";
            case "Low":
                return "Shy, reserved, avoids social interaction.";
            default:
                return "No specific conclusion.";
        }
    }
}

```

```

class Agreeableness extends Trait {
    public Agreeableness(int score) {
        super( name: "Agreeableness (A)", score);
    }

    @Override
    public String getDescription() {
        switch (getRiskLevel()) {
            case "High":
                return "Friendly, empathetic, cooperative.";
            case "Low":
                return "Competitive, self-centered, less empathetic.";
            default:
                return "No specific conclusion.";
        }
    }
}

```

Each class:

- Calls `super()` with its trait name.
- Implements the `getDescription()` method with descriptions specific to that trait.

3. Class: Person

```
class Person {
    String name;
    Trait[] traits;

    public Person(String name, String traitResult) {
        this.name = name;
        this.traits = parseTraits(traitResult);
    }
}
```

name: The person's name.

traits: An array of 5 Trait objects (O, C, E, A, N).

Constructor takes a name and trait string (e.g., "O70-C30-E60-A96-N10").

```
private Trait[] parseTraits(String result) {
    Trait[] t = new Trait[5];
    t[0] = new Openness(Integer.parseInt(result.substring(result.indexOf("O") + 1, result.indexOf("-C"))));
    t[1] = new Conscientiousness(Integer.parseInt(result.substring(result.indexOf("C") + 1, result.indexOf("-E"))));
    t[2] = new Extraversion(Integer.parseInt(result.substring(result.indexOf("E") + 1, result.indexOf("-A"))));
    t[3] = new Agreeableness(Integer.parseInt(result.substring(result.indexOf("A") + 1, result.indexOf("-N"))));
    t[4] = new Neuroticism(Integer.parseInt(result.substring(beginIndex: result.indexOf("N") + 1)));
    return t;
}
```

Parses the trait string using substring methods.

Creates an array of Trait objects based on extracted scores.

```
public void displayAssessment() {
    System.out.println("Assessment for " + name + ":");
    for (Trait t : traits) {
        System.out.println(t.getScore() + " " + t.getName() + ": " + t.getRiskLevel() + " - " + t.getDescription());
    }
}

public boolean isHighRisk() {
    boolean lowC = traits[1].getRiskLevel().equals("Low");
    boolean highN = traits[4].getRiskLevel().equals("High");
    boolean lowE = traits[2].getRiskLevel().equals("Low");
    return lowC || highN || (lowE && highN);
}

public String getName() {
    return name;
}
}
```

Displays all trait scores and descriptions for this person.

Implements the **risk detection logic** from the prompt:

- Rule b: Low C
- Rule c: High N

- Rule d: Low E and High N

4. Main Class: BigFiveAssessment

```
public class BigFiveAssessment {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<Person> people = new ArrayList<>();

        System.out.print("Enter number of people: ");
        int n = Integer.parseInt(scanner.nextLine());

        for (int i = 0; i < n; i++) {
            System.out.print("Enter name: ");
            String name = scanner.nextLine();
            System.out.print("Enter Big Five result (format: 070-C30-E60-A96-N10): ");
            String result = scanner.nextLine();
            people.add(new Person(name, result));
        }

        System.out.print("\nSelect a person to view personality: ");
        String selected = scanner.nextLine();
        for (Person p : people) {
            if (p.getName().equalsIgnoreCase(selected)) {
                p.displayAssessment();
            }
        }

        System.out.println("\n--- High Risk Candidates ---");
        for (Person p : people) {
            if (p.isHighRisk()) {
                System.out.println(p.getName());
            }
        }
    }
}
```

```
class Neuroticism extends Trait {  
    public Neuroticism(int score) {  
        super( name: "Neuroticism (N)", score);  
    }  
  
    @Override  
    public String getDescription() {  
        switch (getRiskLevel()) {  
            case "High":  
                return "Emotionally unstable, anxious, moody.";  
            case "Low":  
                return "Emotionally stable, calm, resilient.";  
            default:  
                return "No specific conclusion.";  
        }  
    }  
}
```